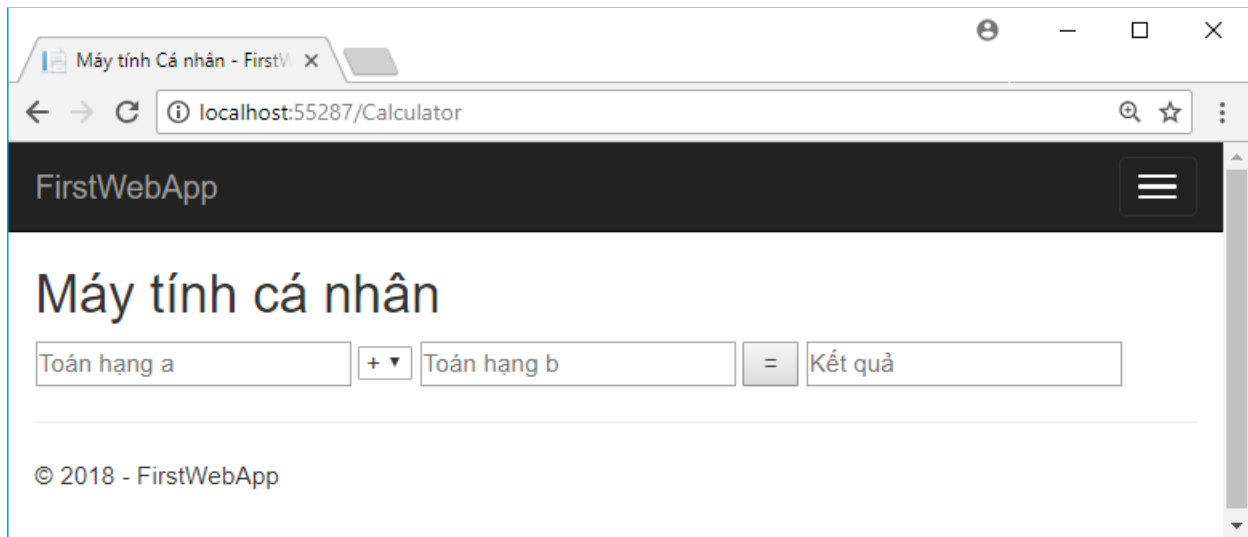


Lab 02: MVC

1 Máy tính cá nhân

1.1 MÔ TẢ

Xây dựng ứng dụng cho phép thực hiện các phép tính đơn giản như cộng, trừ, nhân và chia có giao diện như hình sau. Khi người dùng nhập các toán hạng và chọn toán tử thực hiện sau đó nhấp nút [=] thì chương trình sẽ thực hiện phép tính và hiển thị kết quả lên ô nhập [Kết quả].



Để hoàn thiện bài này, bạn cần thực hiện các bước sau :

Bước 1: Tạo controller **CalculatorController**

Bước 2: Xây dựng giao diện

Bước 3: Thêm action **Calculate** để thực hiện phép tính

Bước 4: Hiển thị kết quả

1.2 Thực hiện

Bước 1: Tạo controller **CalculatorController**

```
public class CalculatorController : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}
```

Bước 2: **Xây dựng giao diện**

Phải chuột lên action Index() để thêm giao diện cho action này và viết mã Razor như sau:

```

1  @{
2      ViewData["Title"] = "Máy tính Cá nhân";
3      Layout = "~/Views/Shared/_Layout.cshtml";
4  }
5
6  <h2>Máy tính cá nhân</h2>
7
8  <form asp-action="Calculate" asp-controller="Calculator" method="post">
9      <input name="a" placeholder="Toán hạng a" />
10     <select name="op">
11         <option>+</option>
12         <option>-</option>
13         <option>x</option>
14         <option>:</option>
15     </select>
16     <input name="b" placeholder="Toán hạng b" />
17     <input type="submit" value="=" />
18     <input placeholder="Kết quả" value="@ViewBag.KetQua" />
19 </form>

```

Giao diện gồm form có action gọi đến action Calculate() của controller CalculatorController và truyền cho action này 3 tham số a (toán hạng a), b (toán hạng b) và op (toán tử op). Đồng thời form này cũng hiển thị giá trị của thuộc tính ViewBag.KetQua lên ô nhập kết quả.

Bước 3: Thêm action **Calculate** để thực hiện phép tính

Để xử lý form, bạn cần bổ sung action Calculate() vào controller CalculatorController để tiếp nhận tham số thực hiện việc tính toán sau đó truyền kết quả về form này để hiển thị kết quả.

```

public ActionResult Calculate(double a = 0, double b = 0, char op = '+')
{
    switch (op)
    {
        case '+':
            ViewBag.KetQua = a + b;
            break;
        case '-':
            ViewBag.KetQua = a - b;
            break;
        case 'x':
            ViewBag.KetQua = a * b;
            break;
        case ':':
            ViewBag.KetQua = a / b;
            break;
    }
    return View("Index");
}

```

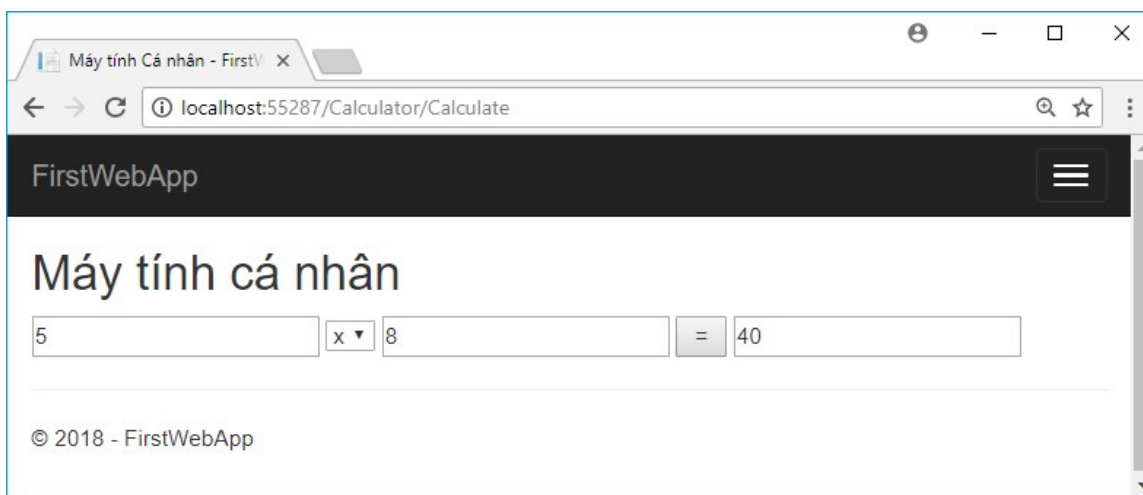
Ở đây chúng ta sử dụng phương pháp nhận tham số bằng đối số action. Như vậy các trường form a, b và op được chuyển vào các đối số của action và tự động chuyển đổi sang kiểu phù hợp. Việc còn lại là xét xem toán tử được chọn là gì để thực hiện phép toán.

Kết quả thực hiện được lưu vào thuộc tính động `KetQua` của đối tượng `ViewBag` để được truyền cho view sử dụng sau này.

Với dòng lệnh `return View("Index")` ở cuối action thì View `Index.cshtml` được chọn để hiển thị. Trong form của view này có dòng mã HTML là `<input placeholder="Kết quả" value="@ViewBag.KetQua" />` do đó kết quả sẽ được hiện thị vào đúng ô kết quả.

Bước 4: **Hiển thị kết quả**

Giao diện sau được thực hiện sau khi nhập 5 và a và 8 vào b và chọn toán tử là x sau đó nhấp nút [=] 40 sẽ hiển thị lên ô kết quả.



2 Generate view from Model

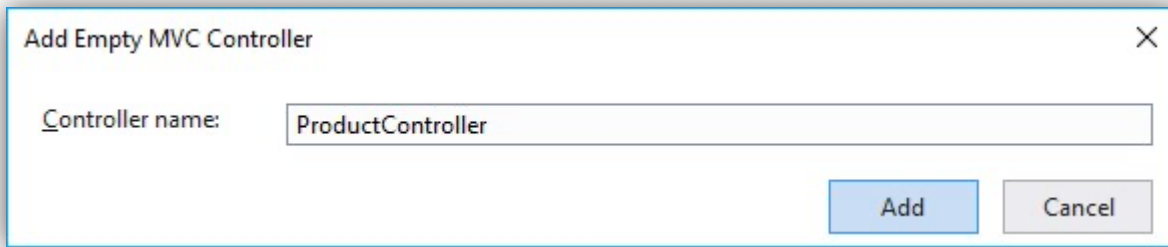
2.1.1 Thêm Model

Thêm mới model **Product** gồm có 3 thuộc tính.

```
public class Product
{
    0 references | 0 exceptions
    public int ID { get; set; }
    0 references | 0 exceptions
    public string Name { get; set; }
    0 references | 0 exceptions
    public double Price { get; set; }
}
```

2.1.2 Tạo mới Controller

New mới Controller dạng Empty, sau đó điền tên Controller là Product.



Tạo xong có sẵn action Index():

```
public class ProductController : Controller
{
    0 references | 0 requests | 0 exceptions
    public IActionResult Index()
    {
        return View();
    }
}
```

Thêm mới action ShowAll():

```
public IActionResult ShowAll()
{
    ViewData["Heading"] = "All Products";
    var products = new List<Product>();
    products.Add(new Product { ID = 101, Name = "IPad 2018", Price = 499 });
    products.Add(new Product { ID = 202, Name = "IPhone X", Price = 999 });
    products.Add(new Product { ID = 303, Name = "SS Note 9", Price = 1099 });
    return View(products);
}
```

2.1.3 Thêm View

Thêm View cho action ShowAll():

@model List<FirstMVCAApp.Models.Product>

```
<html>
<body>
    <h1>@ViewData["Heading"]</h1>
    <table border="1">
        <tr>
            <td>ID</td>
            <td>Name</td>
            <td>Price</td>
        </tr>
```

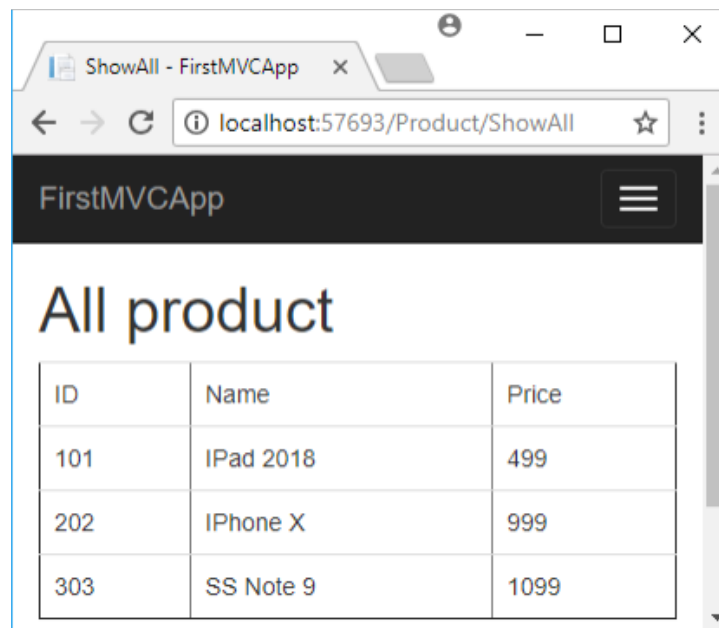
```

@foreach (var p in Model)
{
    <tr>
        <td>@p.ID</td>
        <td>@p.Name</td>
        <td>@p.Price</td>
    </tr>
}
</table>
</body>
</html>

```

2.1.4 Chạy thử nghiệm

<http://localhost:5000/Product/ShowAll>



2.2 Thử các template sẵn có (Create, Edit, List, ...)

Trong controller, khai báo biến tĩnh giữ danh sách các Product.

```
static List<Product> products = new List<Product>();
```

2.2.1 Hiện thị Danh sách

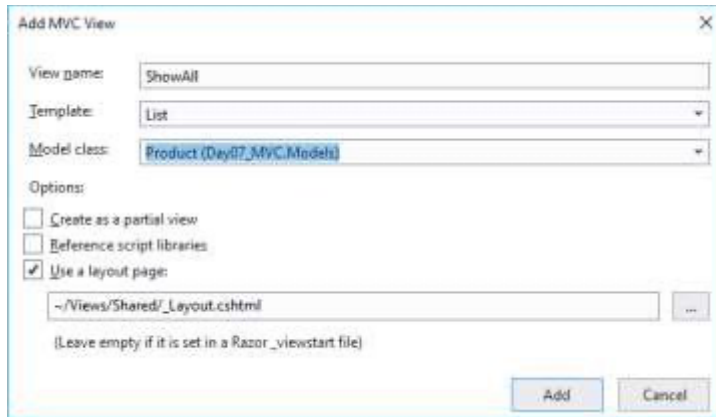
Tạo action:

```

public IActionResult ShowAll()
{
    return View("ShowAll", products);
}

```

Thêm View cho action:



Chỉnh sửa view theo ý người dùng.

Bổ sung các link sửa/xóa:

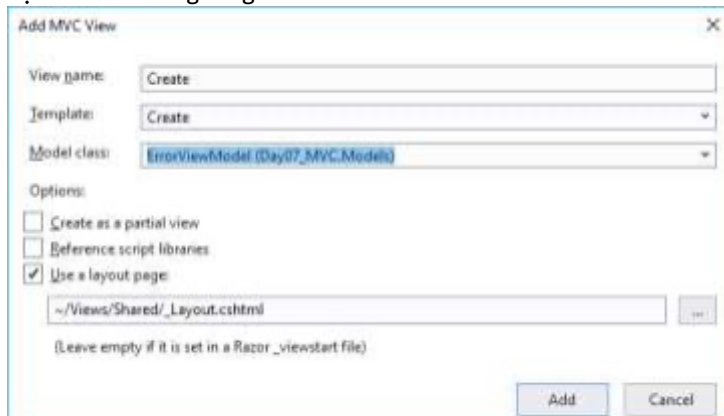
```
<a asp-controller="Product" asp-action="Edit" asp-route-id="@item.Id">Sửa</a> <a
asp-controller="Product" asp-action="Delete" asp-route-id="@item.Id">Xóa</a>
```

2.2.2 Thao tác thêm

Thêm action Create:

```
public IActionResult Create()
{
    return View();
}
```

Tạo View tương ứng:



Thực hiện lưu khi thêm:

[HttpPost]

```
public IActionResult Create([Bind("Id", "Name", "Price")] Product product)
{
    //thêm vào danh sách
    products.Add(product);
    //gọi hiển thị danh sách
    return RedirectToAction("ShowAll");
}
```

2.2.3 Thao tác sửa

Tạo action sửa:

```

public IActionResult Edit(int id)
{
    Product p = products.SingleOrDefault(q => q.Id == id);
    if (p != null) //tìm thấy
    {
        return View(p);
    }
    else
    {
        return NotFound();
    }
}

```

Tạo View:

Xử lý sửa:

```

[HttpPost]
public IActionResult Edit(int id, [Bind("Id", "Name", "Price")] Product product)
{
    //Sửa vào danh sách
    Product p = products.SingleOrDefault(q => q.Id == id);
    if (p != null) //tìm thấy
    {
        p.Name = product.Name;
        p.Price = product.Price;
    }
    //gọi hiển thị danh sách
    return RedirectToAction("ShowAll");
}

```

2.2.4 Xử lý xóa

Có thể tự thêm màn hình confirm.

```

public IActionResult Delete(int id)
{
    //tìm Product cần xóa (dùng LINQ)
    Product p = products.SingleOrDefault(q => q.Id == id);
    if (p != null) //tìm thấy
    {

```

```
        products.Remove(p);  
    }  
  
    //gọi hiển thị danh sách  
    return RedirectToAction("ShowAll");  
}
```

2.2.5 Chạy thử và nhận xét

